PRÁCTICA 3 (sesiones prácticas 3ª y 4ª) Estructuras de Datos y Algoritmos

Título: Implementación lineal en memoria dinámica de TAD genéricos. Curso: 2026/27

Objetivos:

- Implementar en C++ un TAD genérico "*colección de ítems temáticos fechados",* en memoria dinámica y basándose en estructuras lineales.
- Implementar en C++ un TAD no genérico *anuncio*.
- Implementar un programa principal que, utilizando una "*colección de ítems temáticos fechados"* para gestionar una colección de *anuncios*, nos permita probar automáticamente las operaciones de los TAD, leyendo datos y órdenes (a realizar sobre la colección) de un fichero de texto y generando un fichero de texto con el resultado del seguimiento de dichas órdenes.

Fecha límite de entrega: 17-11-2026 (incluido)

Material (descargar de Moodle)

- Ejemplos de ficheros de texto, con los formatos de ficheros de entrada y salida para el programa a desarrollar en la tarea 3.

Descripción detallada:

Este enunciado se utilizará para las sesiones 3ª y 4ª, y además se reutilizará en las prácticas posteriores. En la página 5 se encuentra una propuesta de reparto del trabajo a realizar a lo largo de las semanas disponibles para realizar esta práctica 3.

Se trata de implementar un TAD genérico *"colección de ítems temáticos fechados"*, particularizarlo y utilizarlo con un TAD no genérico *anuncio*, para implementar un programa que los utilice para gestionar una colección de anuncios.

Tienes que realizar las siguientes **tareas**:

1. Implementar en C++ el TAD *anuncio* que se especifica a continuación:

```
espec Anuncios
usa cadenas, fechas[1]
género anuncio {Los valores del TAD anuncio representarán tuplas formadas como:
  (identificador, título, descripción, categoría, publicación, finalización), siendo publicación y
  finalización sendas fechas (válidas según el calendario gregoriano), y el resto sendas cadenas.}

operaciones

  crearAnuncio: cadena ID, cadena t, cadena descrip, cadena categ, fecha publi, fecha fi → anuncio
  {Devuelve un anuncio compuesto con identificador ID, título t, descripción descrip, categoría
   categ, y con fecha de publicación publi y fecha de finalización fi.}
  suIdentificador: anuncio e → cadena
  {Dado un anuncio e formado como (I,T,D,C,P,F) devuelve I, i.e. el identificador del anuncio e.}
  suTítulo: anuncio e → cadena
  {Dado un anuncio e formado como (I,T,D,C,P,F) devuelve T, i.e. el título del anuncio e.}
  cambiarTítulo: anuncio e, cadena n → anuncio
  {Dado un anuncio e formado como (I,T,D,C,P,F) y una cadena n, devuelve un anuncio igual al
   compuesto como (I,n,D,C,P,F).}
  suDescripción: anuncio e → cadena
  {Dado un anuncio e formado como (I,T,D,C,P,F) devuelve D, i.e. la descripción del anuncio e.}
  cambiarDescripción: anuncio e, cadena n → anuncio
  {Dado un anuncio e formado como (I,T,D,C,P,F) y una cadena n, devuelve un anuncio igual al
   compuesto como (I,T,n,C,P,F).}
  suCategoria: anuncio e → cadena
  {Dado un anuncio e formado como (I,T,D,C,P,F) devuelve C, i.e. la categoría del anuncio e.}
  cambiarCategoria: anuncio e, cadena n → anuncio
  {Dado un anuncio e formado como (I,T,D,C,P,F) y una cadena n, devuelve un anuncio igual al
   compuesto como (I,T,D,n,P,F).}
  suPublicación: anuncio e → fecha
  {Dado un anuncio e formado como (I,T,D,C,P,F) devuelve P, i.e. la fecha de publicación de e.}
  suFinalización: anuncio e → fecha
  {Dado un anuncio e formado como (I,T,D,C,P,F) devuelve F, i.e. la fecha de finalización de e.}

fespec
```

2. Desarrollar una implementación en C++, **genérica y en memoria dinámica** basándose en una **lista enlazada ordenada**, de un TAD genérico *colecciónITF,* que representa "*colección de ítems temáticos fechados"*.

   La especificación del *TAD colecciónITF* que vamos a considerar es:

[1] La especificación *fechas* definiendo el TAD fecha representando fechas válidas del calendario gregoriano, visto en clase, y con las operaciones que se considere necesarias. Se podrá usar su implementación del material de clase, o hacer una propia.

```
especificación coleccionesITF
  usa booleanos, naturales, cadenas, fechas
parámetro formal
  género ITF {Ítem Temático Fechado}
    operación {Se requiere que el género ITF tenga definidas:}
      identificador: ITF i → cadena{operación que devuelve cadena con el identificador del ítem i}
      tema: ITF i → cadena {operación que devuelve una cadena con el tema del ítem i}
      fecha: ITF i → fecha {operación que devuelve la fecha perteneciente al ítem i}
fpf
género colecciónITF {Los valores del TAD representan colecciones de datos ITF con identificadores
                     no repetidos.}

operaciones
  crear: → colecciónITF
  {Crea una colección de ITF vacía, sin ítems.}
  añadir: colecciónITF c , ITF i → colecciónITF
  {Si en c no hay ningún ítem con identificador igual al de i, devuelve una colección igual a la
   resultante de añadir el ítem i a la colección c. En caso contrario, devuelve una colección
   igual a c.}
  actualizar: colecciónITF c , ITF i → colecciónITF
  {Si en c no hay ningún ítem con identificador igual al de i, devuelve una colección igual a c. En
   caso contrario, devuelve una colección igual a la resultante de sustituir en c el ítem que tenía
   identificador igual al de i, por el ítem i.}
  borrar: cadena ident , colecciónITF c → colecciónITF
  {Si en c hay algún ítem con identificador igual a ident, devuelve una colección igual a la
   resultante de borrar ese ítem de la colección c. En caso contrario, devuelve una colección
   igual a c.}
  está?: cadena ident , colecciónITF c → booleano
  {Devuelve verdad si y sólo si en c hay algún ítem con identificador igual a ident.}
  parcial obtenerITF: cadena ident , colecciónITF c → ITF
  {Si en c hay algún ítem con identificador igual a ident, devuelve dicho ítem.
   Parcial: operación no definida si en c no hay ningún ítem con identificador igual a ident.}
  obsoletos: fecha f , cadena separador , colecciónITF c → cadena
  {Devuelve la cadena resultante de concatenar los identificadores de todos los ítems que hay en c
   con fecha estrictamente anterior a f, y separados entre sí por la cadena
   separador.}
  purgar: fecha f , colecciónITF c → colecciónITF
  {Devuelve una colección igual a la resultante de eliminar de c todos los ítems con fecha
   estrictamente anterior a f.}
  existeTema: cadena t, colecciónITF c → booleano
  {Devuelve verdad si y sólo si en c hay algún ítem i con tema igual a t.}
  delTema: cadena t , cadena separador , colecciónITF c → cadena
  {Devuelve la cadena resultante de concatenar los identificadores de todos los ítems que hay en c
   con tema igual a t, y separados entre sí por la cadena separador.}
  zanjarTema: cadena t , colecciónITF c → colecciónITF
  {Devuelve una colección igual a la resultante de eliminar de c todos los ítems i cuyo tema es
   igual a t.}
  esVacía?: colecciónITF c → booleano
  {Devuelve verdad si y sólo si c no contiene ningún ítem.}
  tamaño: colecciónITF c → natural
  {Devuelve el número de ítems que hay en la colección c.}

  {Las siguientes operaciones constituyen un iterador que permite visitar los elementos de la
   colección de forma que siempre se visiten antes los elementos con identificador anterior (i.e.
   lexicográficamente menor) a los que se visitarán más tarde:}
  iniciarIterador: colecciónITF c → colecciónITF
  {Inicializa el iterador para recorrer los ítems de la colección c, de forma que el siguiente
   ítem a visitar sea el que tiene el identificador (lexicográficamente) menor (situación de no
   haber visitado ningún ítem).}
  existeSiguiente?: colecciónITF c → booleano
  {Devuelve verdad si queda algún ítem por visitar con el iterador de la colección c, devuelve
  falso si ya se ha visitado el último ítem.}
  parcial siguiente: colecciónITF c → ITF
  {Devuelve el siguiente ítem a visitar con el iterador de la colección c.
   Parcial: la operación no está definida si no quedan ítems por visitar (no existeSiguiente?(c)).}
  parcial avanza: colecciónITF c → colecciónITF
  {Prepara el iterador de la colección c para que se pueda visitar el siguiente ítem, entendiendo
   por siguiente aquel ítem con (lexicográficamente) menor identificador de los todavía no
   visitados.
   Parcial: la operación no está definida si no quedan ítems por visitar (no existeSiguiente?(c)).}
fespec
```

Advertencia: nótese que para la implementación del **TAD *colecciónITF* no se requiere optimizar al máximo posible las operaciones** `obsoletos`, `purgar`, `existeTema`, `delTema`, ni `zanjarTema`. Sería factible plantear para este TAD implementaciones cuya representación interna permita optimizar dichas operaciones, mediante estructuras de datos auxiliares para indexar y gestionar la información por temas y/o por fechas. Sin embargo, tales implementaciones requerirían tener en cuenta y resolver más detalles y dificultades (en esta práctica y en la siguiente) que las que se pretenden con el enunciado tal cual está planteado. **No se recomienda intentar** resolver la práctica

planteando **tales implementaciones,** y en cualquier caso optar por ellas **no permitirá obtener más nota, ni excusará comportamientos incorrectos o ineficientes de las implementaciones entregadas para su evaluación.**

3. Utilizar la implementación del TAD *anuncio*, implementado en la tarea 1, y la del TAD genérico *colecciónITF* implementado en la tarea 2, para implementar un programa de prueba que gestione una colección de anuncios. Los anuncios de la colección se identificarán por su *identificador*. La *categoría* del anuncio será considerada su tema en la colección. La fecha de un anuncio en la colección será la *fecha de publicación* del propio anuncio.

   El programa de prueba deberá crear una colección vacía y, a continuación, leer las instrucciones de las operaciones a realizar con la colección, desde un fichero de texto denominado "entrada.txt". El fichero "entrada.txt" tendrá la siguiente estructura o formato:

```
<instrucción1>
<datos para la instrucción1>
<instrucción2>
<datos para la instrucción2>
...
<instrucciónÚltima>
<datos para la instrucciónÚltima>
```

   Donde `<instrucciónX>` podrá ser alguna de las siguientes cadenas: *"A", "C", "O", "E", "EC", "B", "BC", "P", "LO", "LC", "LT",* que representarán respectivamente las órdenes de: *"**A**ñadir la información de un nuevo anuncio a la colección", "**C**ambiar la información de un anuncio en la colección", "**O**btener toda la información relativa al anuncio de la colección correspondiente a un identificador dado", "comprobar si en la colección **E**xiste algún anuncio con un identificador dado", "comprobar si en la colección **E**xiste algún anuncio con una **C**ategoría igual a la dada", "**B**orrar el anuncio de la colección con un identificador dado", "**B**orrar de la colección todos los anuncios con una **C**ategoría igual a la dada", "**P**urgar la colección, es decir, borrar todos los anuncios con fecha de publicación estrictamente anterior a una dada", "**L**istar los identificadores de todos los anuncios **O**bsoletos de la colección, es decir, aquellos con fecha de publicación estrictamente anterior a una dada", "**L**istar los identificadores de todos los anuncios de la colección de una **C**ategoría igual a la dada", y "**L**istar los detalles relativos a **T**odos los anuncios de la colección, por orden lexicográfico de su identificador*".

   Supondremos que el fichero "entrada.txt" tendrá siempre una estructura como la descrita, sin errores, es decir:
   - Si la instrucción es *"A"* o *"C",* las siguientes 6 líneas del fichero contendrán los datos para la información de un anuncio:
     - una línea con el identificador.
     - una línea con el título.
     - una línea con la descripción.
     - una línea con la categoría.
     - una línea con la fecha de publicación.
     - una línea con la fecha de finalización.
   - Si la instrucción es *"O"*, *"E"* o *"B",* la siguiente línea del fichero contendrá el identificador de anuncio respecto al cual se actúa o consulta.
   - Si la instrucción es *"EC"*, *"BC"* o *"LC",* la siguiente línea del fichero contendrá la categoría de los anuncios respecto a la cual se actúa o consulta.
   - Si la instrucción es *"P"* o *"LO"*, la siguiente línea del fichero contendrá el valor de la fecha a utilizar.
   - Si la instrucción es *"LT"* no requiere que le sigan otras líneas con más datos para su ejecución, así que la siguiente línea sería la de la siguiente instrucción, o el final del fichero.

   Para todas ellas, tanto en el caso de *identificadores* como en el caso de *títulos*, *descripciones* o *categorías*, ninguno de dichos datos superará una línea, pero pueden contener cualquier número de palabras. Las fechas tendrán siempre el formato: *aaaa-mm-dd*, donde *a*, *m* y *d* representan respectivamente los correspondientes dígitos del año, el mes y el día.

   Para resolver cada instrucción, el programa deberá utilizar las operaciones ofrecidas por los TAD implementados en las tareas anteriores. Cuando se procese una instrucción *"A"*, el programa intentará introducir en la colección un nuevo anuncio, creado utilizando los datos dados para la instrucción en el fichero. Cuando se procese una instrucción *"C",* el programa intentará actualizar un anuncio existente en la colección, utilizando los datos dados en el fichero para la instrucción. Cuando se procese una instrucción *"O"*, el programa intentará obtener toda la información relativa al anuncio registrado en la colección con el identificador dado. Cuando se procese la instrucción *"E"*, el programa comprobará si en la colección existe un anuncio con el identificador dado. Cuando se procese una instrucción *"B"*, el programa deberá eliminar de la colección el anuncio con el identificador igual al dado. Cuando se procese la instrucción *"EC"*, el programa comprobará si en la colección existe algún anuncio con categoría igual a la dada. Cuando se procese una instrucción *"BC"*, el programa deberá eliminar de la colección todos los anuncios cuya categoría sea igual a la dada. Cuando se procese una instrucción *"LC"*, el programa deberá listar los identificadores de todos los anuncios existentes en la colección cuya categoría sea igual a la dada, separados entre sí por la cadena "`:::`".

Cuando se procese la instrucción *"P"*, el programa deberá eliminar de la colección todos los anuncios cuya fecha de publicación sea estrictamente anterior a la dada. Cuando se procese una instrucción *"LO"*, el programa deberá listar los identificadores de todos los anuncios existentes en la colección cuya fecha de publicación sea estrictamente anterior a la dada, separados entre sí por la cadena "`:::`". Cuando se procese una instrucción *"LT"*, el programa deberá listar en su salida la información sobre todos los anuncios que contiene la colección, por orden lexicográfico creciente según sus identificadores. El programa finalizará cuando haya procesado todas las instrucciones del fichero.

Como resultado de su ejecución, el programa creará un fichero de texto "salida.txt" en el que irá escribiendo el resultado de cada orden ejecutada, y que constará de las siguientes líneas:

- Por cada instrucción de tipo *"A"*, **si es posible introducir los datos de un nuevo anuncio en la colección**, se escribirá una línea en el fichero de salida que empiece con la cadena "`INTRODUCIDO: `"; **si no es posible introducir el anuncio en la colección**, se escribirá una línea en el fichero de salida que empiece con la cadena "`NO INTRODUCIDO: `". En cualquiera de los dos casos, se seguirá escribiendo a continuación, y en la misma línea del fichero, la concatenación de los datos utilizados para el anuncio, y en el siguiente orden: identificador, título, categoría, descripción, fecha de publicación y fecha de finalización (separados entre sí por la cadena "`:::`", y con las fechas escritas en formato: *aaaa-mm-dd*).

- Por cada instrucción de tipo *"C"*, **si es posible actualizar en la colección los datos del anuncio correspondiente al identificador dado**, se escribirá una línea en el fichero de salida que empiece con la cadena "`CAMBIADO: `", seguida de la concatenación de todos los datos relativos a la nueva información del anuncio en la colección, en la misma línea del fichero y en el siguiente orden: identificador, título, categoría, descripción, fecha de publicación y fecha de finalización (separados entre sí por la cadena "`:::`", y con las fechas escritas en formato: *aaaa-mm-dd*).
  Si **no es posible actualizar el anuncio de dicho identificador en la colección**, se escribirá una línea en el fichero de salida que empiece con la cadena "`NO CAMBIADO: `", seguida del identificador dado para la instrucción.

- Por cada instrucción de tipo *"O",* **si existe** algún anuncio en la colección con el identificador dado, se escribirá una línea que empiece con la cadena "`LOCALIZADO: `", seguida de la concatenación de todos los datos de la información del anuncio localizado en la colección, en la misma línea del fichero y en el siguiente orden: identificador, título, categoría, descripción, fecha de publicación y fecha de finalización (separados entre sí por la cadena "`:::`", y con las fechas escritas en formato: *aaaa-mm-dd*).
  Si no existía ningún anuncio en la colección con identificador igual al dado, se escribirá en el fichero una línea que empiece con la cadena "`NO LOCALIZADO: `", seguida del identificador dado para la instrucción.

- Por cada instrucción de tipo *"E",* **si existe** algún anuncio en la colección con un identificador igual al dado, se escribirá una línea en el fichero que empiece con la cadena "`EXISTENTE: `". **Si no existía** ningún anuncio en la colección con un identificador igual al dado, se escribirá en el fichero una línea que empiece con la cadena "`DESCONOCIDO: `". En cualquiera de los dos casos, se seguirá escribiendo a continuación, y en la misma línea del fichero, el identificador dado para la instrucción.

- Por cada instrucción de tipo *"EC"*, **si existe** algún anuncio en la colección con una categoría igual a la dada para la instrucción, se escribirá una línea que empiece con la cadena "`CATEGORIA CONOCIDA: `", y **si no existía** ningún anuncio en la colección con categoría igual a la dada, se escribirá en el fichero de salida una línea con la cadena "`CATEGORIA DESCONOCIDA: `". En cualquiera de los dos casos, se seguirá escribiendo a continuación, y en la misma línea del fichero, la categoría dada para la instrucción.

- Por cada instrucción de tipo *"B"*, **si existía** algún anuncio en la colección con un identificador igual al dado y este es borrado de la colección, se escribirá una línea que empiece con la cadena "`BORRADO: `", y **si no existía** ningún anuncio en la colección con un identificador igual al dado o este no es borrado de la colección, se escribirá en el fichero una línea que empiece con la cadena "`NO BORRADO: `". En cualquiera de los dos casos, se seguirá escribiendo a continuación, y en la misma línea del fichero, el identificador dado para la instrucción.

- Por cada instrucción de tipo *"BC",* **si existían** anuncios en la colección con categoría igual a la dada, se escribirá una línea que empiece con la cadena "`ELIMINADOS: `", **si no existía** ningún anuncio en la colección con categoría igual a la dada, se escribirá una línea que empiece con la cadena "`NO ELIMINADOS: `". En ambos casos, la cadena escrita irá seguida a continuación, y en la misma línea del fichero, por la categoría dada para la instrucción.

- Por cada instrucción de tipo *"P",* **si existían** anuncios en la colección con fecha de publicación estrictamente anterior a la dada, se escribirá una línea que empiece con la cadena "`PURGADOS: `", **si no existía** ningún anuncio en la colección con fecha de publicación estrictamente anterior a la dada, se escribirá una línea que empiece con la cadena "`NO PURGADOS: `". En ambos casos, la cadena escrita irá seguida a continuación, y en la misma línea del fichero, por la fecha utilizada en la operación (formato: *aaaa-mm-dd*).

- Por cada instrucción de tipo *"LO"*, se escribirá una línea que empiece con la cadena "`ANTERIORES A: `" seguida de la fecha utilizada en la operación (con formato: *aaaa-mm-dd*), seguida de la cadena "` --- `", seguida por la concatenación de los identificadores de los anuncios de la colección con fecha de publicación estrictamente anterior a la dada, separados entre sí por la cadena "`:::`". Para finalizar, se concatenará otra vez la cadena "` --- `".

- Por cada instrucción de tipo *"LC"*, se escribirá una línea que empiece con la cadena "`DE CATEGORIA: `" seguida de la categoría utilizada con la operación, seguida de la cadena "` --- `", seguida por la concatenación de los identificadores de los anuncios de la colección con categoría igual a la dada, separados entre sí por la cadena "`:::`". Antes de finalizar la línea se concatenará otra vez la cadena "` --- `".

- Por cada instrucción de tipo *"LT"*, se escribirá una primera línea con la cadena "`-----LISTADO: `", seguida del número total de anuncios registrados en la colección, y a continuación, por cada anuncio almacenado en la colección, y siguiendo el orden (lexicográfico) creciente según el identificador, se escribirá otra línea con la información de todos los datos relativos al anuncio en la colección, de acuerdo al formato que se describe a continuación:
  `[ identificador -de-> categoría ;;; título ] (publicado) --- descrip --- (finaliza)`

  siendo `identificador`, `categoría`, `título` y `descrip`, respectivamente el identificador, la categoría, el título y la descripción del anuncio, y siendo `publicado` y `finaliza` respectivamente las fechas de publicación y de finalización del anuncio, con dichas fechas escritas en formato: *aaaa-mm-dd*. El listado finalizará con una línea con la cadena "`-----`".

La implementación de la orden *"LT"*, deberá utilizar obligatoriamente las operaciones del iterador de la implementación del TAD *colecciónITF*.

Al final de este documento se muestran los formatos de los ficheros con un ejemplo de fichero "entrada.txt", y su correspondiente fichero "salida.txt". **Los ficheros que se leen y se escriben deberán respetar escrupulosamente los formatos que se describen en el enunciado, o la práctica no será evaluada.**

Detalles sobre la implementación con C++:

- Ejemplo de un esquema básico, de lectura de flujo y de línea a línea, que puede servir para leer el fichero de instrucciones:

```
ifstream f;
f.open("entrada.txt");
string instruccion; string salto;
while (f >> instruccion) {
    getline(f,salto);
    if (instruccion == "A") {
        //... sigue el programa
    } else if (instruccion == "O") {
        //... sigue el programa
    }//... sigue el programa
}//... sigue el programa
```

Propuesta de reparto de trabajo entre las dos sesiones y semanas de la práctica 3:

Recuerda que las prácticas de EDA tienen planificado tiempo que hay que dedicar **a trabajar en ellas antes, durante y después de la sesión de prácticas presencial**. Desde la publicación del enunciado hasta la fecha límite de entrega de la práctica, las dos sesiones de prácticas se distribuyen, para todos los grupos de laboratorio, en un periodo de algo más de cuatro semanas lectivas. Aquí se sugiere una posible división del trabajo entre semanas, para tener una guía de aproximadamente qué parte de la práctica deberías tener hecha y totalmente acabada, antes de la segunda mitad de esta práctica.

- **En las dos primeras semanas:** Implementar y probar completamente el TAD *anuncio*. Del TAD *colecciónITF*, tener en cuenta todas las operaciones para elegir la representación interna en memoria, pero de momento únicamente implementar y probar el correcto funcionamiento de las operaciones: *crear*, *tamaño, esVacía?, añadir, está?, obtenerITF, existeTema, delTema,* y todas las operaciones del *iterador*. Del programa de la tarea 3, implementar solo lo necesario para el correcto funcionamiento de las instrucciones *"A"*, *"E"*, *"O", "EC", "LC"* y *"LT"*.

- **Semanas tercera y cuarta:** Implementar todas las demás operaciones del TAD *colecciónITF*, y probar a fondo el correcto funcionamiento de todas las operaciones del TAD. Del programa de la tarea 3, terminar su implementación y probar a fondo el correcto funcionamiento de todas las instrucciones del programa. Asegurarse de cumplir escrupulosamente con todos los formatos indicados para los ficheros de "entrada.txt" y "salida.txt".

Observaciones.

- **El código fuente de las prácticas evaluables será compilado y probado en *lab000.unizar.es*, que es donde deberá compilar y funcionar correctamente.** Es recomendable que te asegures de que se podrá compilar y ejecutar correctamente en *lab000* dedicando a ello el tiempo y las pruebas suficientes.
- **El código deberá compilar correctamente con la opción** `-std=c++11` **activada**.
  - Esto significa que, si se trabaja con la línea de comandos, deberá compilarse con:

    ```
    g++ -std=c++11  *.cpp
    ```

- **No se impondrá ninguna ruta concreta para los ficheros de código fuente a compilar, ni para los ficheros que leerá/escribirá el programa**. De esta forma, el fichero de entrada será siempre **"entrada.txt"** (no "entrada2.txt", "datos/entrada.txt" o similares), y el fichero de salida se llamará **"salida.txt"**, no "salida2.txt", "misalida.txt", etc.
- Todos los ficheros con código fuente deberán estar **correctamente documentados**.
- En el **comentario inicial de cada fichero** de código fuente se añadirán los **nombres completos y NIP[2] de las personas autoras de dicho código fuente.**
- La salida debe cumplir escrupulosamente las especificaciones del enunciado. Por ejemplo, cuando se indica que en el fichero de salida se escribirá "`NO INTRODUCIDO: `", está escrito en mayúsculas, con un espacio en blanco tras ':', y lo mismo para el resto de instrucciones. **Es obligatorio cumplir todos los formatos descritos en este enunciado, o la práctica no será evaluada**.
- **Los TAD deberán implementarse siguiendo las instrucciones dadas en las clases y prácticas de la asignatura, no se permite utilizar Programación Orientada a Objetos, y tampoco el uso o lanzamiento de excepciones.**
- **No se permite usar** las clases o componentes de la Standard Template Library (STL), ni otras bibliotecas similares, y tampoco se permite utilizar *Smart Pointers*. Tampoco se permite utilizar código ajeno, es decir, no implementado **íntegra y personalmente** por las personas integrantes del equipo de prácticas.

Entrega de la práctica. Instrucciones.

- **La práctica solo deberá entregarla uno de los miembros del equipo de prácticas**.
- La práctica deberá ser entregada en la tarea correspondiente en el curso Moodle, y antes del plazo límite establecido. **No se aceptarán entregas fuera de plazo, ni entregas por ninguna otra vía que no sea la entrega en la tarea Moodle para esta práctica.**
- Crea un fichero comprimido en formato `Zip` y de nombre `practica3_NIP1_NIP2_NIP3.zip`, que contenga todos los ficheros de código fuente desarrollados para resolver la práctica, y dos ficheros de texto ***entrada.txt*** y ***salida.txt***, con los formatos descritos en el enunciado, pero que sean significativamente diferentes a los proporcionados como ejemplo, y con los que habréis probado la implementación realizada en vuestra práctica. Los ficheros no deben estar distribuidos en subdirectorios, y no debe entregarse ningún fichero de código objeto, ni ejecutables, etc.
- El nombre utilizado para el fichero comprimido: `practica3_NIP1_NIP2_NIP3.zip`, debe contener los NIP de los integrantes del equipo, siendo `NIP1<NIP2<NIP3`. De forma similar, si el trabajo se ha hecho de forma individual o en un equipo formado únicamente por 2 personas, el nombre del fichero deberá ser respectivamente `practica3_NIP1.zip` o `practica3_NIP1_NIP2.zip`, incluyendo explícitamente el NIP de cada una de las personas que ha realizado la práctica.
- A la hora de evaluar la práctica se utilizará tanto el fichero de prueba *entrada.txt* y *salida.txt* que se entreguen, como ficheros de prueba entregados por otros compañeros, o ficheros propios de los profesores.

[2] El NIP, también llamado NIA; es el número de 6 o 7 dígitos que la Universidad de Zaragoza asigna a cada estudiante para su identificación.

entrada.txt

```
LT
C
ID40
Maleta pequeña
Maleta con ruedas, pequeña, buen estado.
En venta
2026-09-20
2026-09-01
A
ID40
Maleta pequeña
Maleta con ruedas, pequeña, buen estado.
En venta
2026-09-20
2026-09-01
A
ID10
Maleta cabina
Compraria maleta cabina tamaño Ryanair
Compra
2026-09-10
2026-11-20
A
ID20
Silla paseo bebe
Vendo silla paseo para bebe
En venta
2026-10-10
2026-10-10
A
ID20
Silla paseo plegable bebe
Vendo silla paseo plegable para bebe
En venta
2026-10-11
2026-10-11
LT
C
ID20
Silla paseo bebe
Vendo silla paseo plegable para bebe. Precio negociable
En venta
2026-10-21
2026-10-21
E
ID25
E
iD20
E
ID20
O
ID30
A
ID30
Mecedora
Regalo mecedora en buen estado
Disponible gratis
2026-10-01
2026-06-11
O
ID20
LT
LO
2026-08-31
LO
2026-10-01
B
ID25
EC
Intercambio
```

(continuación de) entrada.txt

```
EC
En venta
LT
B
ID30
LC
Intercambio
LC
En venta
LC
en venta
LT
P
2026-08-31
BC
Disponible gratis
BC
En venta
LT
A
ID20
Silla paseo bebe
Vendo silla paseo plegable para bebe
En venta
2026-10-11
2026-10-11
LT
P
2026-10-01
LT
```

salida.txt

```
-----LISTADO: 0
-----
NO CAMBIADO: ID40
INTRODUCIDO: ID40:::Maleta pequeña:::En venta:::Maleta con ruedas, pequeña, buen estado.:::2026-09-20:::2026-09-01
INTRODUCIDO: ID10:::Maleta cabina:::Compra:::Compraria maleta cabina tamaño Ryanair:::2026-09-10:::2026-11-20
INTRODUCIDO: ID20:::Silla paseo bebe:::En venta:::Vendo silla paseo para bebe:::2026-10-10:::2026-10-10
NO INTRODUCIDO: ID20:::Silla paseo plegable bebe:::En venta:::Vendo silla paseo plegable para bebe:::2026-10-11:::2026-10-11
-----LISTADO: 3
[ ID10 -de-> Compra ;;; Maleta cabina ] (2026-09-10) --- Compraria maleta cabina tamaño Ryanair --- (2026-11-20)
[ ID20 -de-> En venta ;;; Silla paseo bebe ] (2026-10-10) --- Vendo silla paseo para bebe --- (2026-10-10)
[ ID40 -de-> En venta ;;; Maleta pequeña ] (2026-09-20) --- Maleta con ruedas, pequeña, buen estado. --- (2026-09-01)
-----
CAMBIADO: ID20:::Silla paseo bebe:::En venta:::Vendo silla paseo plegable para bebe. Precio negociable:::2026-10-21:::2026-10-21
DESCONOCIDO: ID25
DESCONOCIDO: iD20
EXISTENTE: ID20
NO LOCALIZADO: ID30
INTRODUCIDO: ID30:::Mecedora:::Disponible gratis:::Regalo mecedora en buen estado:::2026-10-01:::2026-06-11
LOCALIZADO: ID20:::Silla paseo bebe:::En venta:::Vendo silla paseo plegable para bebe. Precio negociable:::2026-10-21:::2026-10-21
-----LISTADO: 4
[ ID10 -de-> Compra ;;; Maleta cabina ] (2026-09-10) --- Compraria maleta cabina tamaño Ryanair --- (2026-11-20)
[ ID20 -de-> En venta ;;; Silla paseo bebe ] (2026-10-21) --- Vendo silla paseo plegable para bebe. Precio negociable --- (2026-10-21)
[ ID30 -de-> Disponible gratis ;;; Mecedora ] (2026-10-01) --- Regalo mecedora en buen estado --- (2026-06-11)
[ ID40 -de-> En venta ;;; Maleta pequeña ] (2026-09-20) --- Maleta con ruedas, pequeña, buen estado. --- (2026-09-01)
-----
ANTERIORES A: 2026-08-31 ---  ---
ANTERIORES A: 2026-10-01 --- ID10:::ID40 ---
NO BORRADO: ID25
CATEGORIA DESCONOCIDA: Intercambio
CATEGORIA CONOCIDA: En venta
-----LISTADO: 4
[ ID10 -de-> Compra ;;; Maleta cabina ] (2026-09-10) --- Compraria maleta cabina tamaño Ryanair --- (2026-11-20)
[ ID20 -de-> En venta ;;; Silla paseo bebe ] (2026-10-21) --- Vendo silla paseo plegable para bebe. Precio negociable --- (2026-10-21)
[ ID30 -de-> Disponible gratis ;;; Mecedora ] (2026-10-01) --- Regalo mecedora en buen estado --- (2026-06-11)
[ ID40 -de-> En venta ;;; Maleta pequeña ] (2026-09-20) --- Maleta con ruedas, pequeña, buen estado. --- (2026-09-01)
-----
BORRADO: ID30
DE CATEGORIA: Intercambio ---  ---
DE CATEGORIA: En venta --- ID20:::ID40 ---
DE CATEGORIA: en venta ---  ---
-----LISTADO: 3
[ ID10 -de-> Compra ;;; Maleta cabina ] (2026-09-10) --- Compraria maleta cabina tamaño Ryanair --- (2026-11-20)
[ ID20 -de-> En venta ;;; Silla paseo bebe ] (2026-10-21) --- Vendo silla paseo plegable para bebe. Precio negociable --- (2026-10-21)
[ ID40 -de-> En venta ;;; Maleta pequeña ] (2026-09-20) --- Maleta con ruedas, pequeña, buen estado. --- (2026-09-01)
-----
NO PURGADOS: 2026-08-31
NO ELIMINADOS: Disponible gratis
ELIMINADOS: En venta
-----LISTADO: 1
[ ID10 -de-> Compra ;;; Maleta cabina ] (2026-09-10) --- Compraria maleta cabina tamaño Ryanair --- (2026-11-20)
-----
INTRODUCIDO: ID20:::Silla paseo bebe:::En venta:::Vendo silla paseo plegable para bebe:::2026-10-11:::2026-10-11
-----LISTADO: 2
[ ID10 -de-> Compra ;;; Maleta cabina ] (2026-09-10) --- Compraria maleta cabina tamaño Ryanair --- (2026-11-20)
[ ID20 -de-> En venta ;;; Silla paseo bebe ] (2026-10-11) --- Vendo silla paseo plegable para bebe --- (2026-10-11)
-----
PURGADOS: 2026-10-01
-----LISTADO: 1
[ ID20 -de-> En venta ;;; Silla paseo bebe ] (2026-10-11) --- Vendo silla paseo plegable para bebe --- (2026-10-11)
-----
```